

Name- Raut Avishkar. H

Kodbud Python Internship Tasks

Complete source code submission for Tasks 1, 2, 5, and 8.

Task 1 - Simple Calculator (CLI)

Task 1 · Simple Calculator (CLI) A command-line calculator that handles addition, subtraction, multiplication, and division. Uses input() for user interaction and if-elif-else for operation routing. Includes division-by-zero protection. Source Code

```
# Task 1: Simple Calculator (CLI)

# Concepts: input(), float conversion, if-elif-else,
arithmetic operators
def calculator():
    print("=" * 40)
    print(" Simple Calculator")
    print("=" * 40)
    print("Operations: + - * /")
    print()
    try:
        num1 = float(input("Enter first
number : "))
        op = input("Enter
operator (+, -, *, /): ").strip()
        num2 = float(input("Enter second
number : "))
    except ValueError:
        print("Invalid input! Please enter
numeric values.")
        return
    if op == "+":
        result = num1 + num2
        symbol = "+"
    elif op == "-":
        result = num1 - num2
        symbol = "-"
    elif op == "*":
        result = num1 * num2
        symbol = "*"
    elif op == "/":
        if num2 == 0:
            print("Error: Division by zero is not
allowed!")
            return
        result = num1 / num2
        symbol = "/"
    else:
        print(f"Unknown operator '{op}'.
Please use +, -, *, or /.")
        return
    print(f" {num1} {symbol} {num2} = {result}")
    print("=" * 40)
    # Entry point
    while True:
        calculator()
        again = input("\nCalculate again?
(yes/no): ").strip().lower()
        if again not in ("yes", "y"):
            print("Goodbye!")
            break
```

Sample Run

```
Enter first number : 15
Enter operator : /
Enter second number : 4
15.0 / 4.0 = 3.75
```

Task 2 - Number Guessing Game

Task 2 · Number Guessing Game

Uses Python's random module to pick a secret number between 1 and 100. The player receives 'Too high' / 'Too low' hints on each guess. Attempt count is tracked and displayed on success.

Source Code

```
# Task 2: Number Guessing Game
# Concepts: random module, while loop, int
conversion, attempt counter
import random
def
guessing_game
():
print("=" * 45) print(" Number
Guessing Game") print("=" * 45)
print("I've picked a secret number
between 1 and 100.") print("Try to
guess it!\n") secret =
random.randint(1, 100) attempts = 0
max_tries = 10 # bonus: limited
attempts while attempts <
max_tries: try:
guess = int(input(f"Attempt {attempts +
1}/{max_tries} - Your guess: ")) except ValueError:
print(" Please enter a
whole number.\n") continue
attempts += 1 if
guess < 1 or guess
> 100:
print(" Out of range! Guess
between 1 and 100.\n") attempts -
= 1 # don't penalise out-of-range
continue if guess == secret:
print() print(f" Correct! The
number was {secret}.") print(f"
You guessed it in {attempts}
attempt(s).") if attempts == 1:
print(" Incredible - first
try!") elif attempts <= 3:
print(" Impressive!")
print("=" * 45) return elif
guess < secret: diff =
secret - guess hint = "Very
close! " if diff <= 5 else
"" print(f" Too low!
{hint}Try higher.\n") else:
diff = guess - secret hint =
"Very close! " if diff <= 5
else "" print(f" Too high!
{hint}Try lower.\n") # Ran
out of attempts print()
print(f" Game over! The secret
number was {secret}.") print("
Better luck next time!") print("="
* 45)
```

```

# Entry
point
while
True:
    guessing_game() play_again =
    input("\nPlay again? (yes/no):
    ").strip().lower() if play_again not in
    ("yes", "y"):
    print("Thanks for
    playing!") break

```

Task 5 - Password Strength Checker

Task 5 · Password Strength Checker Validates a password against five criteria using Python's re (regex) module and string methods. Gives a score-based rating: Weak, Moderate, or Strong, with actionable feedback. Source Code

```

# Task 5: Password Strength Checker #
Concepts: re module, string methods,
conditional logic, f-strings import re
def check_password(password: str) -> dict:
    """ Returns a dict with individual rule
    results and an overall rating.
    Rules:
    1. Minimum 8 characters
    2. At least one uppercase letter (A-Z)
    3. At least one lowercase letter (a-z)
    4. At least one digit (0-9)
    5. At least one special character
    (!@#$%^&*...) """ rules = { "min_length"
    : len(password) >= 8,
    "has_upper" : bool(re.search(r"[A-Z]",
    password)),
    "has_lower" : bool(re.search(r"[a-z]",
    password)),
    "has_digit" : bool(re.search(r"\d",
    password)), "has_special" :
    bool(re.search(r"[!@#$%^&*(),.?\":{}|<>\-
    _+=\[\]\\"/>

```

```

"has_digit" : "At least one digit (0-9)",
"has_special": "At least one special
character",
} print() print(" Password
Analysis:") print(" " + "-"
* 36) for key, label in
labels.items(): status =
"PASS" if rules[key] else
"FAIL" mark = "[+]" if
rules[key] else "[-]"

print(f" {mark}
{label:<34} {status}")
print(" " + "-" * 36)
print(f" Score :
{score}/5") print(f"
Rating : {rating}") if
rating == "Weak":
print(" Tip : Add uppercase, digits, and
special characters.") elif rating ==
"Moderate":
print(" Tip : Almost there - fix the
remaining rules above.") else: print("
Your password is secure!") print() #
Entry point print("=" * 45) print("
Password Strength Checker") print("=" *
45) while True:
pwd = input("Enter a password to check
(or 'quit'): ").strip() if pwd.lower() ==
"quit": print("Exiting checker.") break
if not pwd: print(" Password cannot be
empty.\n") continue display_result(pwd)

```

Task 8 - Simple Chatbot (CLI)

Task 8 · Simple Chatbot (CLI) A rule-based chatbot that matches user input using keyword detection and if-elif-else logic. Handles greetings, FAQs, small talk, and an unknown-input fallback. Includes a quit command to exit cleanly. Source Code

```

# Task 8: Simple Chatbot (CLI)
# Concepts: if-elif-else, string methods,
keyword matching, loops def
get_response(user_input: str) -> str: """
Maps user input to a bot response using
keyword matching.
All comparisons are case-insensitive. """ msg =
user_input.lower().strip() # Greetings if any(word
in msg for word in ["hello", "hi", "hey", "howdy",
"greetings"]):
return "Hello! I'm PyBot. How can I help you today?"
# Farewells elif any(word in msg for word in ["bye",
"goodbye", "see you", "exit", "quit"]):

```

```

return "Goodbye! Have a great day!" # How
are you elif any(phrase in msg for phrase
in ["how are you", "how r you", "how do you
do", "what's up"]):
return "I'm just a bot, but I'm running
perfectly! Thanks for asking." # Name query
elif any(phrase in msg for phrase in ["your
name", "who are you", "what are you called"]):
return "I'm PyBot, a simple Python chatbot built for
the Kodbud internship!" # Python help elif any(word
in msg for word in ["python", "programming", "code",
"coding"]): return ("Python is awesome! It's great for
automation, web dev, data science, " "and AI. What
would you like to know?")
# Internship questions elif any(word in msg for
word in ["internship", "task", "kodbud", "project"]):
return ("This chatbot is part of a Kodbud
Python internship. " "Complete 4 out of 8
tasks and upload them to LinkedIn!")
# Time / date elif any(word in msg for
word in ["time", "date", "today", "day"]):
from datetime import datetime now =
datetime.now() return f"Current date & time:
{now.strftime('%A, %d %B %Y - %I:%M %p')}" #
Jokes elif any(word in msg for word in ["joke",
"funny", "laugh", "humor"]):
return ("Why do programmers prefer
dark mode?\n" "Because light attracts
bugs! :-)")
# Thank you elif any(word in msg for word in
["thank", "thanks", "thx", "appreciate"]):
return "You're welcome! Feel free to ask
me anything else." # Help menu elif
"help" in msg or "?" == msg:

return ("I can chat
about:\n" " -
Greetings & small
talk\n"
" - Python programming\n"
" - Kodbud internship info\n"
" - Current date/time\n"
" - Jokes!\n"
"Just type naturally and I'll do my best.")
#
Fallbac
k else:
return ("Hmm, I didn't quite understand
that. " "Try asking about Python, the
internship, or type 'help'.")
def
run_chatbot(
): print("="
* 50)
print(" PyBot - Simple
Python Chatbot") print("="

```

```
* 50) print("Type 'quit'
or 'bye' to exit.\n")
while True:
user_input = input("You :
").strip() if not
user_input:
continue response =
get_response(user_input)
print(f"PyBot : {response}\n")
# Exit when the bot says goodbye
if any(word in
user_input.lower() for word in
["quit", "bye", "goodbye",
"exit"]):
break
#Entry point run_chatbot() All four programs are complete,
standalone CLI scripts. Run each directly with python filename.py.No
third libraries are required – only Python's standard library.
```